

Trabajo Práctico Unidad 4

Normalización avanzada y redundancia

Integrantes Grupo E: Luciano Nicolás Ibáñez, Xavier Damiano Pappalardo, Lautaro Gómez Cánovas, Maximiliano Reinoso, Martín Sisterna y Benjamín Arrollo.

Repositorio:

https://github.com/LauGomezCanovas/UTN-TP_Base_de_Datos_Semana8_2PRO1/tree/main/UTN-TUPaD-TP_Base_de_Datos_Semana4_2PRO1-main/TP-Semana4-FoodStore_Gomez_Pappalardo_Ibanez_Arroyo_Reinoso_Sisterna/Semana%208

Artefactos: tp_fnbc_control_lote.sql, tp_desnormalizacion_top_categorias.sql y este informe.

Parte 1 — FNBC sobre “control_lote_almacen”

Notación: **L** = LoteID, **D** = DepositoID, **R** = ResponsableControlID; universo $U = \{L, D, R\}$.

Los scripts usan los nombres reales del proyecto (id_usuario, id_lote, id_deposito) en lugar de los id genéricos de la consigna.

a) Dependencias funcionales

- DF1: $\{L, D\} \rightarrow R$. Para un lote y un depósito dados, el responsable queda unívocamente determinado. Es la DF que modela la clave primaria declarada.
- DF2: $R \rightarrow D$. Cada responsable pertenece, como dato maestro, a un único depósito.

No se postulan $L \rightarrow D$ ni $L \rightarrow R$: un lote grande se fracciona y se controla desde varios depósitos, por lo que puede tener varios responsables. Ambas DF se verificaron contra la instancia con consultas GROUP BY ... HAVING COUNT(DISTINCT ...) > 1 que devuelven 0 filas (sección 2 del script).

b) Clausuras y claves candidatas

Subconjunto X	Clausura X^+	Superclave
$\{L\}$	$\{L\}$	No
$\{D\}$	$\{D\}$	No
$\{R\}$	$\{R, D\}$ (por DF2)	No: no alcanza L
$\{D, R\}$	$\{D, R\}$	No: no alcanza L

$\{L, D\}$	$\{L, D, R\}$ (por DF1) = U	Sí. Mínima: $\{L\}^+$ y $\{D\}^+$ no cubren U → clave candidata
$\{L, R\}$	$\{L, R, D\}$ (por DF2) = U	Sí. Mínima: $\{L\}^+$ y $\{R\}^+$ no cubren U → clave candidata

Claves candidatas: $\{LoteID, DepositoID\}$ y $\{LoteID, ResponsableControlID\}$.

Atributos primos: los tres (L, D y R pertenecen a alguna clave).

No primos: ninguno. Como no hay atributos no primos, el esquema está en 2FN y en 3FN (en $R \rightarrow D$, el lado derecho D es primo); justamente por eso la 3FN no detecta el problema y hace falta la definición de FNBC.

c) Verificación de FNBC

Definición: R está en FNBC si y solo si, para toda DF no trivial $X \rightarrow Y$ que se cumple en R, X es superclave.

DF no trivial	Determinante	Superclave
$\{L, D\} \rightarrow R$	$\{L, D\}$	Sí: $\{L, D\}^+ = U$
$R \rightarrow D$	$\{R\}$	No: $\{R\}^+ = \{R, D\} \neq U$

“control_lote_almacen” NO está en FNBC. La dependencia violatoria es $R \rightarrow D$: su determinante es solo una parte de la clave candidata $\{L, R\}$, no una superclave. La instancia lo exhibe: el valor $R = 801$ aparece en dos filas, algo imposible si $\{R\}$ fuera superclave.

SELECT responsable_control_id, COUNT(*) AS filas

FROM control_lote_almacen GROUP BY responsable_control_id HAVING COUNT(*) > 1;

responsable_control_id | filas

-----+-----

801 | 2

d) Anomalías sobre la instancia $\{(501,30,801), (502,30,801), (503,31,802)\}$

Inserción: Se contrata al responsable 803 para el depósito 32, pero todavía no controla ningún lote. El hecho «803 pertenece al 32» no puede registrarse: “lote_id” integra la PK y no admite NULL; Al intentarlo, el motor responde null value in column "lote_id" ... violates not-null constraint; la única salida sería inventar un lote ficticio.

Borrado: El lote 503 es el único que pasó por el depósito 31. Para anular ese control se borra la fila (503, 31, 802) y con ella desaparece el único registro de que 802 pertenece al 31. En la prueba, las filas que ubican a 802 pasan de 1 a 0: se pierde un dato maestro de personal como efecto colateral de borrar un hecho operativo.

Actualización: El responsable 801 figura en dos filas (lotes 501 y 502, depósito 30). Si se lo transfiere al depósito 33 y solo se actualiza una de ellas, el motor lo acepta y 801 queda en dos depósitos a la vez ({30,33} en la prueba), violando DF2. El mismo cambio exige tocar tantas filas como lotes controle el responsable.

e) Descomposición sin pérdida

Se aplica el algoritmo sobre la DF violatoria $R \rightarrow D$: $R1 = X \cup Y = \{R, D\}$ (responsable_deposito, PK = R, que captura DF2 por construcción) y $R2 = U - Y = \{L, R\}$ (control_lote, PK = {L, R}, la clave alternativa del apartado b). Ambas están en FNBC: en R1 el único determinante es R, que es su clave; R2 no tiene DF no triviales. El script completo (tablas, PK, FK, índices de apoyo y la vista v_control_lote_almacen con la reunión natural) está en tp_fnbc_control_lote.sql, sección 4.

```
CREATE TABLE responsable_deposito (
    responsable_control_id BIGINT NOT NULL REFERENCES usuario(id_usuario),
    deposito_id            BIGINT NOT NULL REFERENCES deposito(id_deposito),
    PRIMARY KEY (responsable_control_id));

CREATE TABLE control_lote (
    lote_id                BIGINT NOT NULL REFERENCES lote(id_lote),
    responsable_control_id BIGINT NOT NULL REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id));

CREATE VIEW v_control_lote_almacen AS
```

```
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
FROM control_lote cl JOIN responsable_deposito rd USING (responsable_control_id);
```

f) Reunión sin pérdida y migración verificada

Justificación: $R1 \cap R2 = \{R\}$. En $R1$, $\{R\}^+ = \{R, D\} = R1$, es decir, el atributo común es superclave de $R1$. Por el teorema de Heath (para una descomposición binaria, sin pérdida $\Leftrightarrow R1 \cap R2$ es superclave de $R1$ o de $R2$), $R1 \bowtie R2$ reconstruye exactamente la relación original, sin tuplas perdidas ni espurias.

Migración: Se migra desde la tabla original con `INSERT ... SELECT` (no se vuelven a tipear los datos), después de comprobar que DF2 vale en los datos. Verificación con la vista de compatibilidad:

Prueba (resultado real)	Filas
original EXCEPT vista	0
vista EXCEPT original	0
conteo original / conteo vista	3 / 3

En el esquema descompuesto las tres anomalías desaparecen (sección 6 del script): 803 se registra en el depósito 32 sin lotes, borrar el único control del depósito 31 deja intacto «802 → 31», y transferir a 801 es un único UPDATE de una fila que se refleja en todos sus controles.

La descomposición no preserva DF1

La FNBC garantiza descomposición sin pérdida, pero no siempre preserva dependencias. Tras separar las tablas, $\{L, D\} \rightarrow R$ ya no se expresa como PK o UNIQUE (L vive en control_lote y D en responsable_deposito): nada impediría asignar a un mismo lote dos responsables del mismo depósito. Se lo cubre con dos disparadores (sección 7 del script), probados con tres casos: rechazar un segundo responsable del mismo depósito para un lote (OK, rechazado), permitir que el lote 501 sea controlado también por 802 desde otro depósito (OK, caso de negocio válido) y rechazar un traslado que duplicaría responsables (OK, rechazado).

Parte 2 — Desnormalización controlada: top 5 de categorías del día

Las columnas de la consigna (c.nombre, dp.subtotal, ped.fecha) se adaptaron a las reales del proyecto (nombre_categoria, subtotal_detallepedido, fecha_pedido).

a) Consulta original: EXPLAIN ANALYZE

Instancia: 151.996 pedidos y 303.991 detalles vigentes, con 2.506 pedidos de hoy (la sección 0 del script agrega 150.000 pedidos de los últimos 60 días sobre los datos de la Semana 5).
 Tiempo de ejecución: 118,7 ms (mediana de 3 corridas: 135,1 / 113,5 / 118,7). Nodo dominante: Parallel Seq Scan on “detalle_pedido”, que lee las ~304.000 filas del historial para quedarse con ~5.000 del día, junto con Parallel Seq Scan on pedido (descarta 50.833 filas por proceso). En total toca 6.459 buffers.

Limit (actual time=118.114..118.660 rows=5) Buffers: shared hit=6459

-> Finalize GroupAggregate (Group Key: c.nombre_categoria)

-> Gather Merge (Workers Launched: 2)

-> Partial HashAggregate

-> Hash Join (pr.categoria_id = c.id_categoria)

-> Nested Loop

-> Parallel Hash Join (dp.pedido_id = ped.id_pedido)

-> Parallel Seq Scan on detalle_pedido dp

(actual time=0.019..41.337 rows=101330 loops=3)

-> Parallel Hash

-> Parallel Seq Scan on pedido ped

(actual time=0.105..26.486 rows=835 loops=3)

Rows Removed by Filter: 50833

-> Memoize -> Index Scan using producto_pkey on producto pr

-> Hash -> Seq Scan on categoria c

Execution Time: 118.733 ms

b) Patrón elegido y justificación

Se eligió una tabla resumen “resumen_ventas_categoria_dia” (fecha, categoria_id, total_vendido) mantenida por disparadores, y no una vista materializada. Evidencia: el plan de (a) lee todo el historial (6.459 buffers, ~119 ms) para responder por un solo día, y su costo crece con la base, no con las ventas de hoy; como el panel lo ejecuta muchas veces por minuto, conviene leer a lo sumo una fila por categoría. Una vista materializada no sirve para un reporte «en tiempo real»: queda desactualizada hasta el próximo REFRESH, y cada REFRESH recalcula todo el historial.

Sincronización: Cuatro disparadores actualizan el resumen en la misma transacción que la venta (alta y modificación de “detalle_pedido”, baja o cambio de fecha de pedido, cambio de categoría de producto); las bajas están cubiertas porque “fn_soft_delete” convierte todo DELETE en UPDATE eliminado = TRUE; y la vista “v_auditoria_resumen_ventas” recalcula la verdad desde la fuente para detectar cualquier desvío.

Reversibilidad: Sí, sin pérdida de información: es redundancia pura derivada de las tablas fuente, que no se modifican; quitar disparadores y tabla vuelve al estado original, y la tabla se reconstruye con una sola consulta GROUP BY.

c) Implementación

Disparador	Evento	Efecto sobre el resumen
trg_resumen_detalle_ins	AFTER INSERT en detalle_pedido (por sentencia, tabla de transición)	Suma el subtotal de los detalles vigentes nuevos
trg_resumen_detalle_upd	AFTER UPDATE en detalle_pedido (OLD y NEW TABLE)	Delta = aporte nuevo – aporte viejo (cantidad, baja lógica, cambio de pedido o producto)
trg_resumen_pedido_upd	AFTER UPDATE OF fecha_pedido, eliminado en pedido (por fila, con WHEN)	Quita o mueve el aporte de todos los detalles del pedido

trg_resumen_producto_categoria	AFTER UPDATE OF categoria_id en producto (por fila, con WHEN)	Reasigna el historial del producto a la nueva categoría
--------------------------------	---	--

La carga inicial y la activación de los disparadores ocurren en una sola transacción con LOCK TABLE ... SHARE ROW EXCLUSIVE, para que no se cuele ninguna venta entre ambas. Los upserts se emiten ordenados por (fecha, categoría) para evitar deadlocks. Script completo: tp_desnormalizacion_top_categorias.sql, secciones 2 a 6.

d) Consulta sobre la estructura desnormalizada y comparación

```
SELECT c.nombre_categoria AS categoria, r.total_vendido
FROM resumen_ventas_categoria_dia r
JOIN categoria c ON c.id_categoria = r.categoria_id
WHERE r.fecha = CURRENT_DATE
ORDER BY r.total_vendido DESC LIMIT 5;
```

Limit (actual time=0.025..0.026 rows=5)

Buffers: shared hit=10

-> Sort (top-N quicksort) -> Hash Join

-> Seq Scan on categoria c (6 filas)

-> Hash -> Bitmap Heap Scan on resumen_ventas_categoria_dia r (rows=5)

-> Bitmap Index Scan on resumen_ventas_categoria_dia_pkey

Execution Time: 0.038 ms

	Antes (4 tablas)	Después (tabla resumen)
Tiempo de ejecución (mediana de 3)	118,7 ms	0,044 ms
Nodo dominante	Parallel Seq Scan on detalle_pedido (+ Parallel Seq Scan on pedido)	Bitmap Index Scan sobre la PK del resumen (5 filas)
Filas leídas de la tabla grande	~304.000 (detalle_pedido)	5
Buffers	6.459	10
Mejora	—	$\approx 2.700\times$ en tiempo; $\approx 650\times$ en buffers

Equivalencia de resultados: original EXCEPT resumen = 0 filas y resumen EXCEPT original = 0 filas.

Contraste honesto: agregar solo un índice sobre pedido(fecha_pedido) baja el plan original de ~115 a ~91 ms en la misma corrida, pero la consulta sigue leyendo y sumando todos los detalles del historial (Parallel Seq Scan sobre “detalle_pedido”, ~50 ms); el índice se probó y se descartó dentro de una transacción con ROLLBACK.

e) Auditoría de sincronización

“v_auditoria_resumen_ventas” recalcula (fecha, categoría, total) desde “detalle_pedido”, pedido y producto y lo compara con el resumen mediante un FULL OUTER JOIN; cualquier fila devuelta es un desvío. Sobre la base ya migrada devuelve 0 filas. Se la probó, además, contra operaciones reales y contra una corrupción deliberada (cada prueba en BEGIN ... ROLLBACK):

Prueba	Operación	Desincronizaciones
T1	Alta con sp_crear_pedido (total de hoy +13.500)	0
T2	UPDATE de cantidad en un detalle de hoy	0
T3	Baja de detalle vía DELETE (soft delete)	0
T4 / T5	Baja de pedido: detalles primero, o pedido solo	0 / 0
T6	Cambio de fecha_pedido (de hoy a ayer)	0
T7	Un producto cambia de categoría (historial reasignado)	0
T8	Alta masiva: 1.000 pedidos y 1.000 detalles en una sentencia	0
T9 (negativa)	Se suman 100 al resumen a mano	1 (detectada)
Auditoría final	Base migrada y sin pruebas pendientes	0 filas

Costo y límites del patrón

- Costo en escritura: en una carga de 1.000 detalles el disparador del resumen tarda ~3,9 ms de ~80 ms totales ($\approx 5\%$); en un alta de un solo detalle (sesión recién abierta) ronda 1,1–1,3 ms de ~4 ms.
- Fila caliente: todas las ventas de una categoría en un día actualizan la misma fila; bajo mucha concurrencia esas escrituras se serializan.
- Baja de pedido y detalle en una misma sentencia (por ejemplo un CTE con dos UPDATE) no está cubierta: se probó y deja 2 desvíos, que la auditoría detecta. El flujo del proyecto (HU-PED-04, dos UPDATE en una transacción) sí lo está. TRUNCATE tampoco.

- Semántica: el resumen replica exactamente la consulta de la consigna, que no filtra por estado; los pedidos CANCELADOS suman. Si el negocio quisiera excluirlos, hay que sumar el estado a la fuente de verdad y a los disparadores.